Python Crash Course

November 23, 2025

Python is a valuable addition to your Linux toolkit. Bash and shell scripting is good for glueing things together and controlling your Linux computer, but a more powerful language like Python is essential to make your computer do interesting things. A major project in this class is that we will make a website using the Django web framework. Before we do that, let's make sure you know the absolute basics of how Python works.

---

1 Assignment

Recreate the programs from this assignment.

1. Section 4.1 - data_types.py
2. Section 4.2 - if_else.py
3. Section 4.3 - list.py
4. Section 4.5 - for_loops_2.py
5. Section 4.6 - while_loops.py
6. Section 4.7 - function_one.py
7. Section 4.7 - function_two.py
8. Section 4.8 - point.py
9. Section 6.2 - datascience.py
10. Section 6.3 - numpy_example.py

If you want to learn more after this assignment, you can go on google or youtube and find a million tutorials.

2 Quick overview

- Python is whitespace sensitive
- Python is whitespace sensitive
- Python is whitespace sensitive
- Did I mention that Python is whitespace sensitive?

Whitespace sensitive means that Python uses indentation to separate logical blocks of code.

Compare the Java and Python programs in figure 1 and figure 2 . You do not need to write these programs. Notice how both do the same thing, but Java uses curly braces ({ and }) whereas Python does not. As such, Python code is often much shorter and more readable than code in other languages.

```
root@droplet:~/python_tutorial# cat for_loop.py
#!/usr/bin/python3

for i in range(3):
    print(i)
root@droplet:~/python_tutorial# ./for_loop.py
0
1
2
```

Figure 1: Python code is very short and doesn't have any { or }. This program prints 0, 1, 2.

```
root@droplet:~/python_tutorial# apt install openjdk-21-jdk
openjdk-21-jdk is already the newest version (21.0.9+10-1~deb13u1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
root@droplet:~/python_tutorial# cat ForLoop.java
public class ForLoop{
        public static void main(String[] args)
        {
                for(int i = 0; i < 3; i++)
                {
                        System.out.println(i);
                }
        }
}
root@droplet:~/python_tutorial# javac ForLoop.java
root@droplet:~/python_tutorial# java ForLoop
0
1
2
```

Figure 2: Whereas the Python code used indentation, Java uses curly braces like { and } to separate blocks of code. As such, Java programs are much bigger than Python programs. This program does the same thing as the Python program: it prints 0, 1, 2.

3 Install python

It should already be installed on Debian 13. On other distros you might have to do something like this:

```
root@droplet$ apt install python3
```

4 The Essentials

In this section we will look at basic essential Python syntax so you know how the language works.

4.1 Variables and datatypes

The basic data types in Python are

- **int** - whole numbers like 1, 4, -100
- **float** - decimal number like 1.32, -0.9
- **string** - text like "hello!"
- **bool** - True or False

See figure 3 for a short script that uses these data types. See figure 4

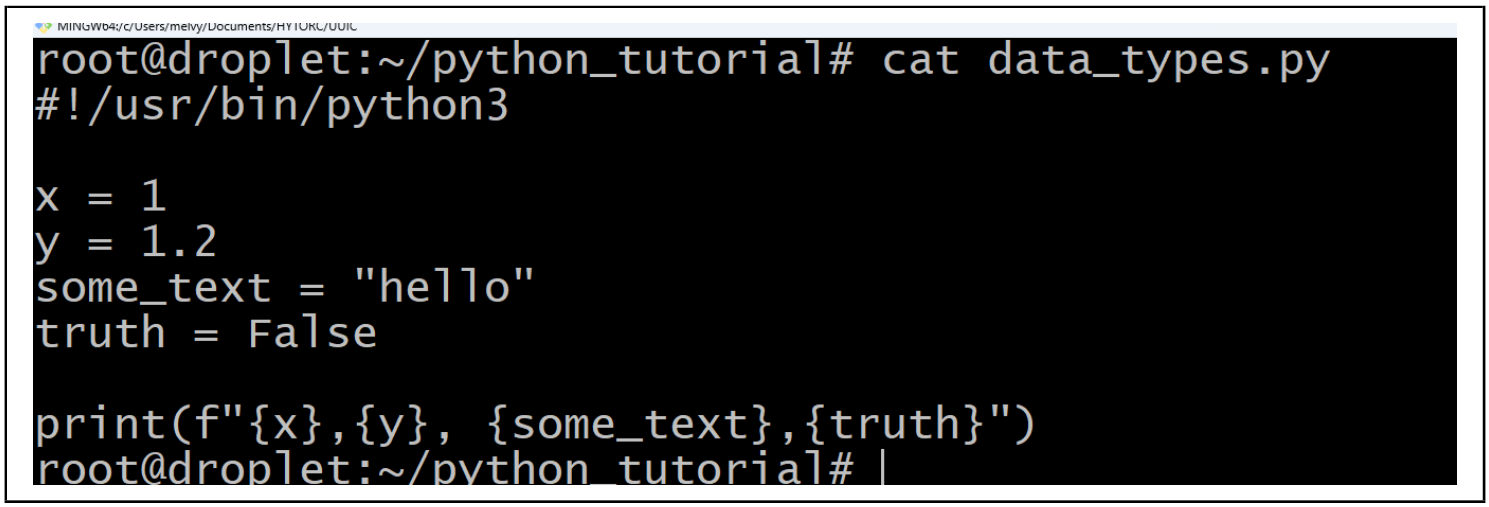

Figure 3: `This little program creates some variables with various data types and prints them out.`

4.2 if / else

Python can run different bits of code depending on whether or not a condition is true. See figure 5

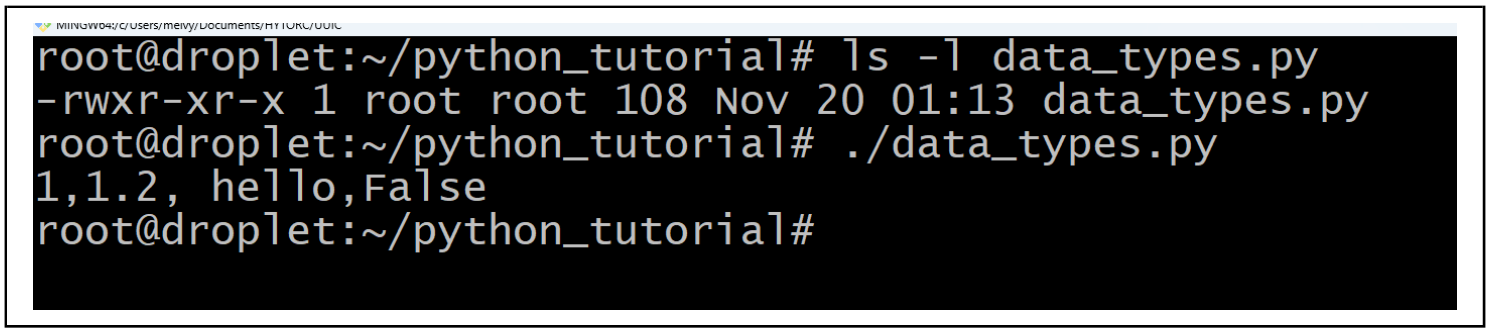

Figure 4: Use chmod to make the program executable. Then you can run it.

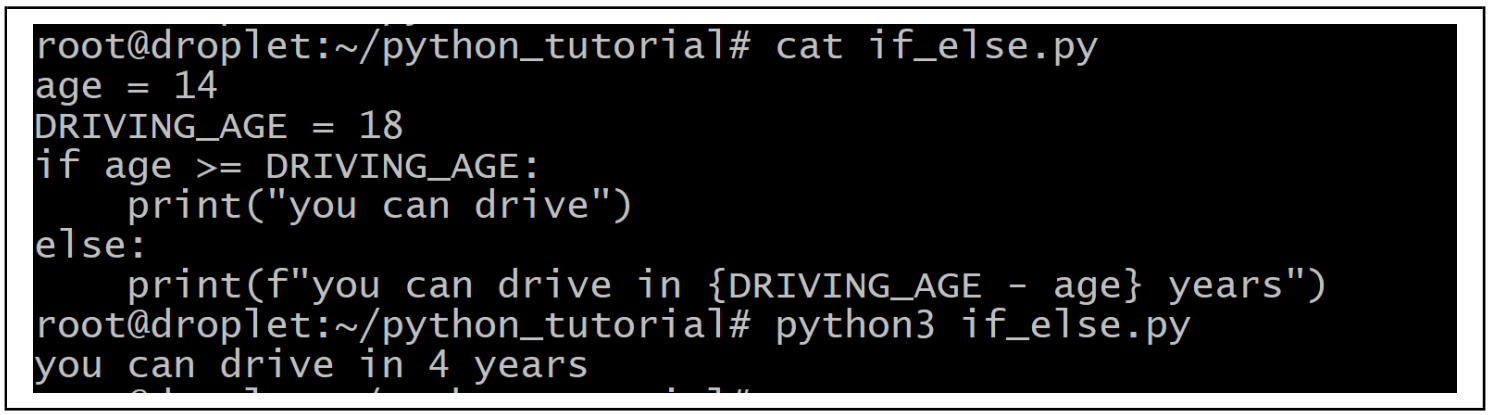

Figure 5: Note that I ran this program by saying python3 if_else.py instead of running it as an executable program like ./if_else.py. You can run python programs either way. If you want to run the program as an executable, you need to make sure you put the shebang line #!/usr/bin/python3 at the top of the file, and you need to use chmod to make it executable.

4.3 lists

Python can make lists of things. You might be familiar with Arrays in other progamming languages. It's pretty much the same thing. See figure 6 to see how you make a list of numbers, and how to print out the first element from the list.

After you run this program, you should create your own list with different values. You can put and of the data types we discussed in section 4.1 like integers, floats, bools, and strings.

```
root@droplet:~/python_tutorial# cat list.py
myList = [1, 2, 10, 8]

print(myList[0])
root@droplet:~/python_tutorial# python3 list.py
1
```

Figure 6: `As in many other languages, the first element in the list is the "0th" element, not the "1st". That's why I print out myList[0] instead of myList[1].`

4.4 Other containers

Besides lists, there are other containers like

- tuple
- set
- dictionary

Feel free to research these as they are very important.

4.5 for loop

Python has loops that can loop over a range of numbers, or the elements in a list. Have a look at figures 1 and 7 to see two examples of using a for loop.

```
root@droplet:~/python_tutorial# cat for_loop_2.py
myList = ["hello", "goodbye"]

for word in myList:
    print(word)
root@droplet:~/python_tutorial# python3 for_loop_2.py
hello
goodbye
```

Figure 7: In this example, we loop over the elements of a list. Compared to a for loop in Java or C++, for loops in Python are very compact and easy to write. Just be careful with the whitespace!

4.6 while loop

A while loop is a loop that keeps going over and over and over so long as a condition is true. In this example, we start with number 3, and do a countdown until it's zero. Then we say happy new year! Have a look at figure 8.

```
root@droplet:~/python_tutorial# cat while_loop.py
num = 3
print("Countdown!")
while num > 0:
    print(f"{num}!")
    num -= 1
print("Happy new year!")
root@droplet:~/python_tutorial# python3 while_loop.py
Countdown!
3!
2!
1!
Happy new year!
```

Figure 8: `3! 2! 1! Happy New Year! Should auld aquaintance be forgot...`

4.7 functions

Functions are the bread and butter of programming. You write little logical blocks inside functions and call them when you want. Functions are important in Python, Bash, Java, and all programming languages I can think of. Functions in python programs come in 2 flavors:

1. They can do something. See figure 9
2. They can do something, and then return a value to you. See figure 10

Functions in python are defined with the **def** keyword, and the contents of the function are indented. Python is whitespace sensitive after all. If you want to return a value from a function, you use the **return** keyword. Have a look at the examples to understand better.

```
root@droplet:~/python_tutorial# vim function_one.py
root@droplet:~/python_tutorial# cat function_one.py
def say_hi(name):
    print(f"hi, {name}!")

say_hi("John")
say_hi("Jane")
say_hi("Everybody")
root@droplet:~/python_tutorial# python3 function_one.py
hi, John!
hi, Jane!
hi, Everybody!
root@droplet:~/python_tutorial#
```

Figure 9: This function takes a person's name and then says hi and that's it. It doesn't return any values.

```
root@droplet:~/python_tutorial# vim function_two.py
root@droplet:~/python_tutorial# cat function_two.py
def add(x, y):
    return x + y

result = add(1, 2)
print(f"The sum is {result}")
root@droplet:~/python_tutorial# python3 function_two.py
The sum is 3
root@droplet:~/python_tutorial#
```

Figure 10: This function adds 2 numbers and returns the sum.

4.8 classes

Python is an amazing language. It can even do **object oriented programming** a.k.a **OOP**, which is a wildly popular programming paradigm. Some languages like Java are strictly object oriented programming languages - everything is a class. Python is more flexible. You can use classes in Python if you want, but it's not requried.

Classes are used for creating more interesting data types, beyond the data types we discussed in section 4.1. In figure 11 you see a little class I wrote called **Point** that represents a coordinate, like the ones you use in math class. We compute the distance between the points using the distance formula from algebra class: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

```
import math

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(0, 0)
p2 = Point(3, 4)

distance = math.sqrt((p2.x - p1.x)**2 + (p2.y - p1.y)**2)
print(f"The distance between these points is {distance}")

~
~
~
```

Figure 11: `This is point.py. Can you guess what it prints out? NOTE: The init function has TWO underscores before init and TWO underscores after init. __init__`

.

```
root@droplet:~/python_tutorial# vim point.py
root@droplet:~/python_tutorial# python3 point.py
The distance between these points is 5.0
root@droplet:~/python_tutorial#
```

Figure 12: `Here is the output of point.py`

.

4.9 And that's it!

There is certainly much more to know about Python, but these essential building blocks are the same as in all the languages you've likely used before. If you wrote and ran these examples, you have a little taste of what Python code looks like and how it acts.

5 Practical Considerations

Python is a very beginner friendly language. Except configuring Python can be frustrating. Every operating system and platform tries to make it "easy" for you, and they all fail.

Configuring Python is painful.

The real problem is with Python *packages*. In section 4 you learned the basics of using Python, but if you want to start writing real software you're going to need to install some packages. We already used a package above in section 4.8 when you did **import math** and then you did the sqrt by calling **math.sqrt**. The algorithm to compute a square root is very complicated, so we use the math package to do it for us.

But configuring python packages is a very painful and sad experience. Want to see an example? To install python packages you use the **pip** command. In this class we will use the Django package to make a website. Go ahead and try to **python3 -m pip install django**. See figures 13 and 14 to see the errors you get, along with a description of the errors. Debian deliberately breaks the whole pip experience so you can't get into the messy situation. The proper way to use Python on debian is with virtual environments, and that's the topic of section 5.1.

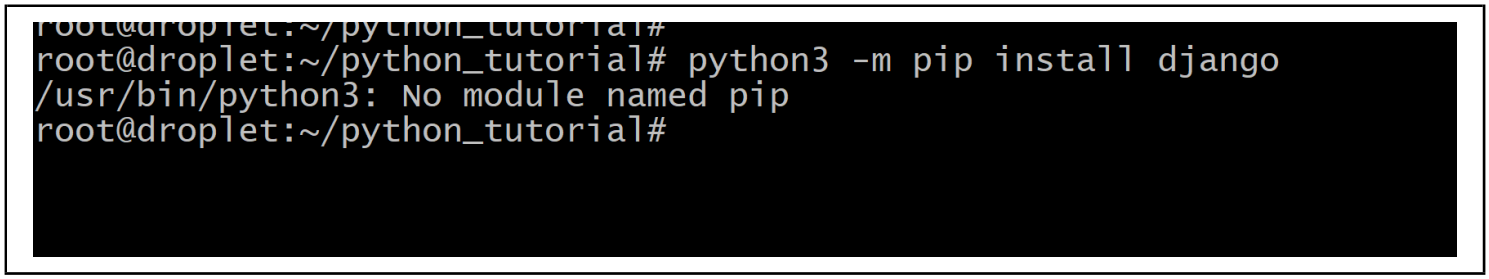

Figure 13: `Debian doesnt give you pip, probably because they know that it's going to cause problems.`

Python is wonderful. Installing packages in python is a painful experience if you don't use a venv.

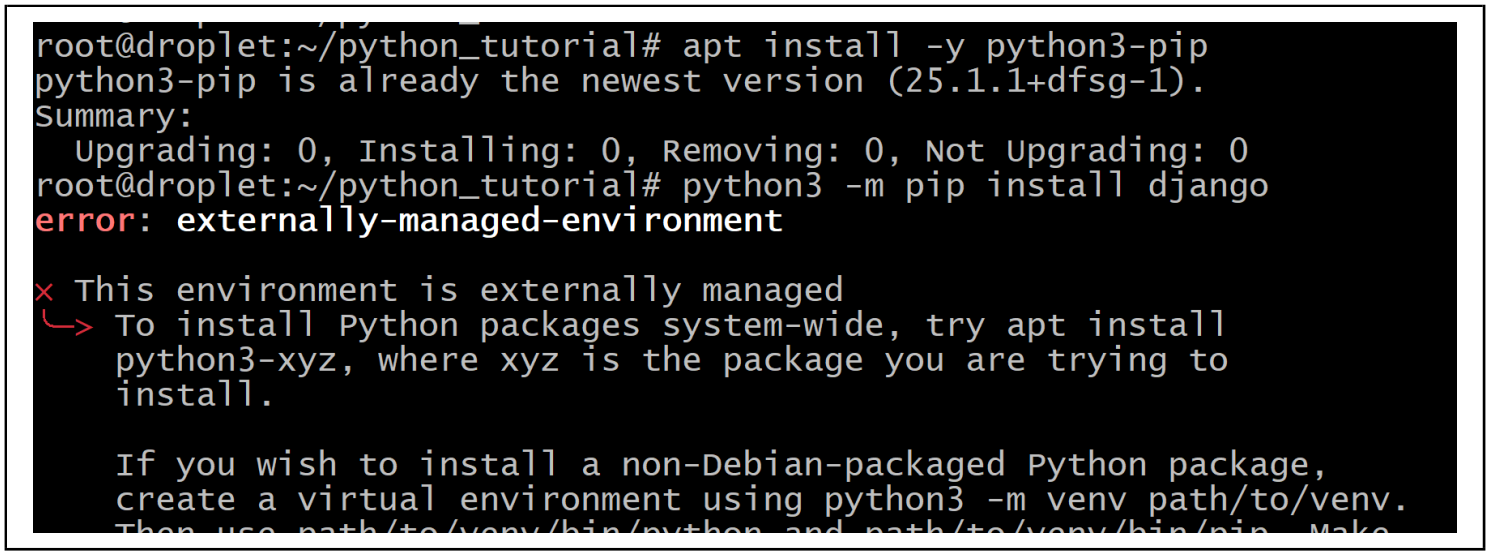

Figure 14: But you think you're smarter than debian so you go ahead and apt install -y python3-pip. Looks what happens if you try to use it! You get a big long error message and they yell and you and tell you to use a virtual environment.

5.1 virtual environments

Virtual environments give you a little "sandbox" that is separate from the system python. You can install whatever python packages you want in this environment, and they are only installed in THAT environment. This helps prevent errors. Have a look at figure 15 to see how to create an activate a virtual environment! You can activate your environment with the **source** command and you deactivate it with the **deactivate** command (see figure 16 to see how to deactivate). If your computer reboots, or you close your ssh connection, you'll need to reactivate the environment.

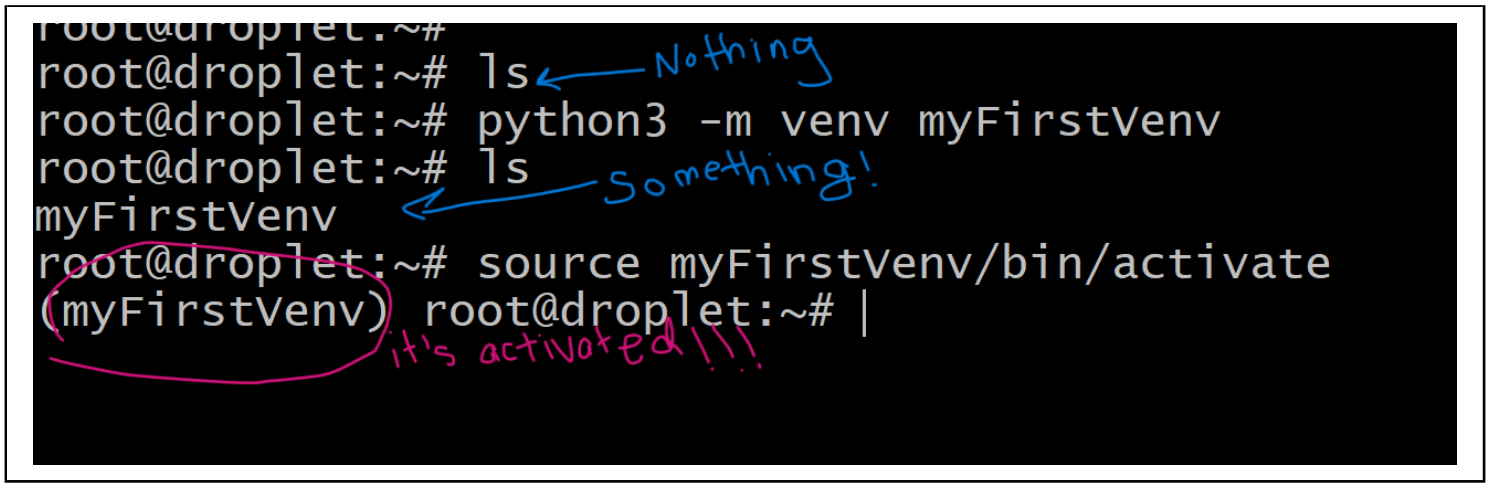

Figure 15: You create the environment by saying python3 -m venv NAME. You activate the environment with source NAME/bin/activate. NAME can be whatever you want to name your virtual environment. If I'm working on a django project, usually I give a nice name like myDjangoVenv or something like that. Note that when its activated your prompt changes. In the example it says (myFirstVenv) after it's activated.

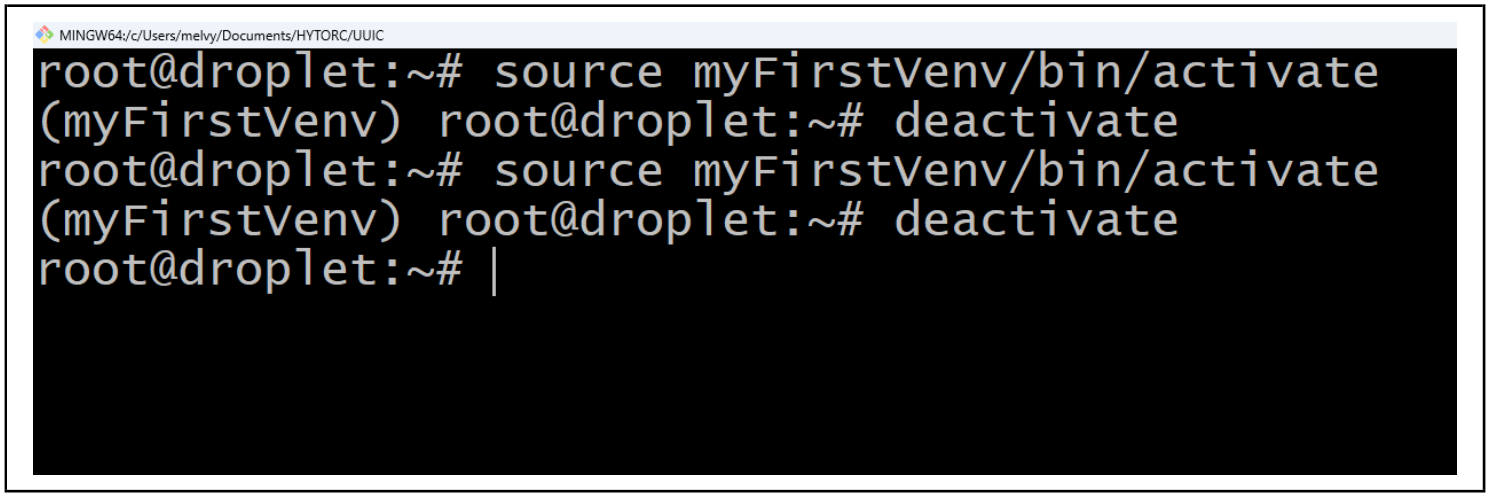

Figure 16: Deactivate the venv with the deactivate command. Reactivate it with the source command.

Now that you have a venv set up, activate it, and we can start installing and using packages without the headache of using the Debian python packages.

6 Some Useful Packages

You can do virtually anything with Python. Many people are using Python for everything from game development to data science. In this section we'll take a quick look at how you install and use some packages in your virtual environment.

6.1 Shebang warning!

You'll notice that I'm usually calling scripts by saying **python3 SCRIPT_NAME.py**. This is because the shebang line we used before **#!/usr/bin/python3** looks for **python3** to be in the **/usr/bin** directory. But we're using a virtual environment now!! We're not using that system python3.

The recommended shebangline to put is: **#/usr/bin/env python3**. But even using this shebang line can potentially lead to problems if you're not careful. So to keep things easy, we're not putting a shebang line at all. If you want to use the recommended shebang line, then make a script as shown in listing 1.

Listing 1: Correct shebang line

```
1 #!/usr/bin/env python3
2
3 print("I'm using whatever python3 is in my PATH!")
4 print("This could be the system python3, or the python3 from the virtualenv!")
```

6.2 pandas

Pandas is a widely used and beloved package for datascience (Pandas, like the bamboo-eating bears). You can use pandas for analyzing large data sets. Install pandas as shown in figure 17, then create the script shown in figure 18 and run it as shown in figure 19. Congratulations! You're written your first python program that uses Pandas!

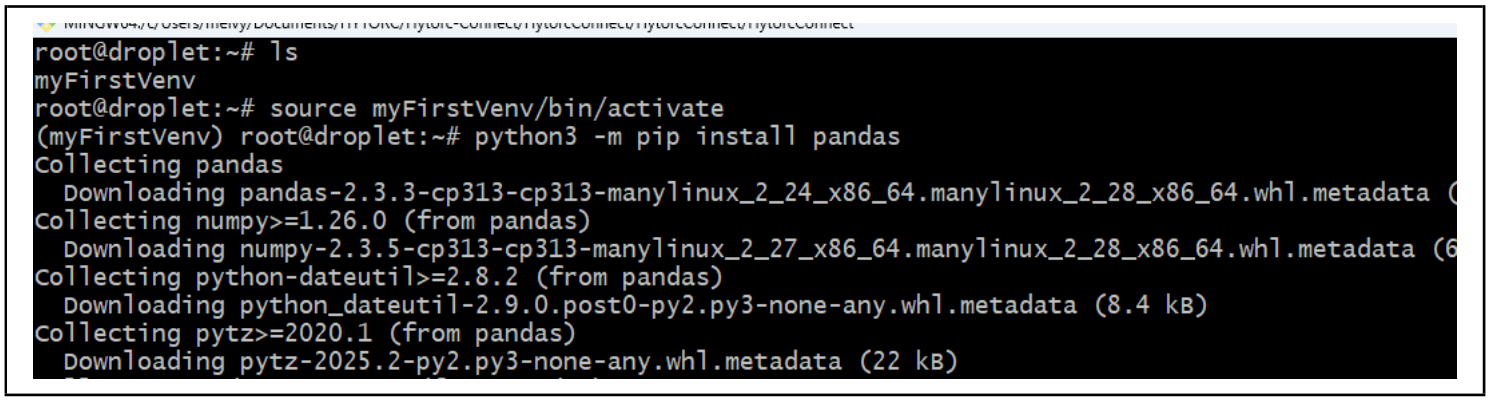

Figure 17: `How to install pandas`

MINGW64:/c/Users/melvy/Documents/HYTORC/Hytorc-Connect/HytorcConnect/HytorcConnect/HytorcConnect

```
import pandas as pd

data = {
    "food eaten" : ["hamburger", "candy", "vegetable"],
    "calories": [420, 380, 5]
}

df = pd.DataFrame(data)
print(df)
totalCalories = df["calories"].sum()
print(f"You ate {totalCalories} calories")
~
~
~
~
```

Figure 18: `Script to add up all the calories you consumed based on a dataset of what you ate.`

```
(myFirstVenv) root@droplet:~# vim datascience.py
(myFirstVenv) root@droplet:~# python3 datascience.py
  food eaten  calories
0  hamburger       420
1      candy       380
2  vegetable         5
You ate 805 calories
(myFirstVenv) root@droplet:~#
```

Figure 19: `Program output.`

6.3 numpy

Numpy is a blazing-fast, powerful package for doing vector and matrix math (linear algebra) in Python. Fields like computer graphics and artificial intelligence are almost entirely based on vector and matrix calculations performed at incredible speeds. If you need to implement a fancy algorithm in python, you'll almost certainly use numpy.

It's definitely worth mentioning for the young, aspiring computer scientists in this class: If you're planning on implementing algorithms instead of just consuming them, you'll need to study linear algebra... and taking as many math classes as possible wouldn't hurt. Virtually all of the amazing things you see in video games or AI are the result of huge linear algebra calculations.

Install numpy as shown in figure 20, write the program shown in figure 21 and then run it to see the output shown in figure 22.

Figure 20: `Install numpy.`

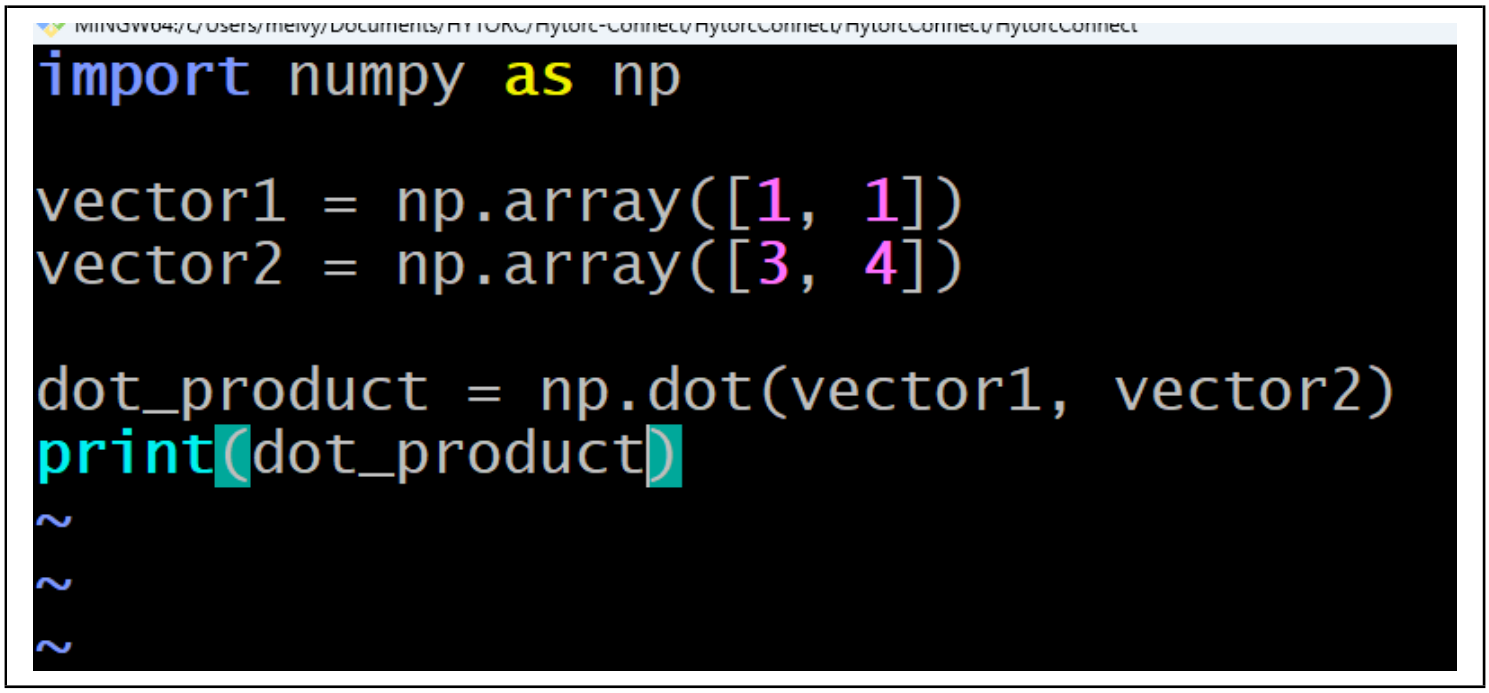

Figure 21: `A small program to compute the dot product of 2 vectors. The output should be 7.`

```
(myFirstVenv) root@droplet:~# vim numpy_example.py
(myFirstVenv) root@droplet:~# python3 numpy_example.py
7
(myFirstVenv) root@droplet:~#
```

Figure 22: `Program output.`

7 What's Next

Previously in this class you made a static website using HTML, CSS and JavaScript. In an upcoming lecture you'll combine this knowledge with what you just learned about Python to make basic websites with Django and the Django Rest Framework. These are **backend web frameworks** that allow you to quickly make beautiful and useful dynamic websites.

Our goal is simply to expose you to a variety of useful things you can do on Linux. Don't fret if you're not totally confident in what you've just learned. It takes years to master a programming language. Hope to see you back here soon.